Object Oriented Programming (CSEG1044)
Unit 03 – Lecture 02 Notes
Exception Handling (Core)

Tofik Ali

February 27, 2026

Contents

1 Syllabus Alignment (Unit 03) **2**

2 Prerequisites **2**

3 Learning Outcomes **2**

4 Lecture Overview **2**

5 Key Terms (Quick) **2**

6 Detailed Notes **3**

6.1 Exceptions vs Errors . . . 3
6.2 What happens when an exception occurs? (stack unwinding) . . . 3
6.3 How to read a stack trace (very important skill) . . . 3
6.4 Checked vs unchecked exceptions (when to use which?) . . . 4
6.5 `try/catch/finally` and multiple catch . . . 4
6.6 Full example: `try/catch/finally` with multiple catches . . . 4
6.7 `throw` vs `throws` . . . 7
6.8 Custom exception design (basic, but practical) . . . 8

7 Worked Example (tutorial): build a safe validation flow **9**

8 Worked Example: custom checked exception (recoverable case) **10**

9 In-Class Exercises (with brief solutions) **11**

10 Common bad practices (avoid) **11**

11 Self-Study Practice (no solutions) **11**

12 Summary (what to remember) **11**

13 Checkpoint Questions (with sample answers) **12**

1 Syllabus Alignment (Unit 03)

Unit 03: exception handling and exception handlers

2 Prerequisites

You should already know:

- methods, parameters, return values,
- classes and objects,
- basic input validation (e.g., checking empty strings).

3 Learning Outcomes

After this lecture, you should be able to:

- Distinguish exceptions vs errors; checked vs unchecked.
- Write `try/catch/finally` with multiple catch blocks.
- Explain `throw` vs `throws` and apply them correctly.
- Create a basic custom exception and use it for validation.

4 Lecture Overview

Exceptions are Java's structured way to report **abnormal situations** without immediately crashing a program. Instead of returning special values like `-1` everywhere, Java can:

- stop the normal flow,
- "bubble up" the problem to a place that can handle it,
- and still clean up resources reliably.

In this lecture you will learn what exceptions are, how to handle them, and how to create simple custom exceptions for clearer programs.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

5 Key Terms (Quick)

- **Exception**: abnormal condition a program can potentially handle.
- **Error**: serious problem typically not recoverable (e.g., `OutOfMemoryError`).
- **Checked exception**: must be caught or declared (e.g., `IOException`).

- **Unchecked exception**: subclass of `RuntimeException` (e.g., `NullPointerException`).
- **Stack trace**: call history showing where an exception occurred.
- **Handler**: catch block that responds to an exception.

6 Detailed Notes

6.1 Exceptions vs Errors

- **Exceptions** are conditions that can be handled: invalid input, missing file, etc.
- **Errors** are typically JVM-level problems: memory exhaustion, serious linkage errors, etc.

Rule of thumb. Catch exceptions you can handle meaningfully. Do not catch `Error` in normal application code.

6.2 What happens when an exception occurs? (stack unwinding)

When an exception is thrown:

1. Java creates an exception object (with message + stack trace).
2. The current method stops executing immediately.
3. Java looks for a matching `catch` in the current method.
4. If not found, the exception goes to the caller (method that called it).
5. This continues until a catch is found or the program terminates.

This process is called **stack unwinding**.

6.3 How to read a stack trace (very important skill)

When you see a stack trace, do this:

1. Find the first line: exception type + message.
2. Look for the first stack frame that points to **your code** (your file/class).
3. That line number is usually where the exception was thrown.
4. If you see `Caused by:`, read the **root cause** first.

Do not panic: stack traces are your friend.

6.4 Checked vs unchecked exceptions (when to use which?)

Category	Typical meaning
Checked (`Exception`)	Caller is expected to handle it; condition is reasonably recoverable (e.g., `IOException`).
Unchecked (`RuntimeException`)	Programming mistake or invalid argument; often not recoverable at runtime (e.g., `NullPointerException`).

Practical guideline.

- Use **checked** when the caller can take a meaningful alternate action (try another file, ask user again, retry).
- Use **unchecked** for invalid arguments and broken assumptions (caller passed wrong values).

6.5 `try/catch/finally` and multiple catch

Basic structure:

- `try` contains code that may throw an exception.
- `catch` handles specific exception types.
- `finally` executes whether an exception occurs or not (useful for cleanup).

Best practice:

- catch the most specific exception you can handle,
- avoid `catch (Exception e)` unless you rethrow/log properly,
- never silently swallow exceptions.

Catch order matters. Always catch the more specific exceptions first. If you catch `Exception` first, other catches become unreachable.

Multi-catch (one handler for multiple types).

```java
try {
    // ...
} catch (NumberFormatException | IllegalArgumentException ex) {
    System.out.println("Bad input: " + ex.getMessage());
}
```

Use multi-catch when the handling is identical.

6.6 Full example: `try/catch/finally` with multiple catches

Problem statement: read two integers from the user and print `a/b`. We want the program to:

- handle invalid integer input (e.g., user types `abc`),
- handle divide-by-zero cleanly,

- always perform cleanup (close `Scanner`) and print a final message.

Full code (copy, compile, run)

```java
import java.util.Scanner;

public class DivideDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter a: ");
            int a = Integer.parseInt(sc.nextLine());

            System.out.print("Enter b: ");
            int b = Integer.parseInt(sc.nextLine());

            int q = a / b; // may throw ArithmeticException
            System.out.println("a/b = " + q);

        } catch (NumberFormatException ex) {
            System.out.println("Please enter valid integers.");

        } catch (ArithmeticException ex) {
            System.out.println("Cannot divide by zero.");

        } finally {
            sc.close();
            System.out.println("Cleanup done (finally).");
        }
    }
}
```

Detailed explanation (what each block does)

1) `try` block:

- Reads input using `sc.nextLine()`.
- Converts to integer using `Integer.parseInt(...)`.
- Computes the quotient using `a / b`.

What can go wrong inside `try`?

- If the user types `abc`, then `Integer.parseInt("abc")` throws `NumberFormatException`.
- If the user enters `b = 0`, then `a / 0` throws `ArithmeticException`.

2) `catch` blocks:

- A catch block runs only if an exception of that type (or subtype) occurred in the try block.
- Java chooses the **first matching catch** from top to bottom.

3) `finally` block:

- Runs **after** the try/catch finishes — whether an exception happened or not.
- Used for **cleanup**: closing files, DB connections, scanners, releasing locks.
- It also runs even if you `return` inside `try` or `catch`.

Rare exceptions: `finally` may not run if the JVM exits abruptly (e.g., `System.exit(0)`) or the process is killed.

Sequence of execution (step-by-step)

Students often memorize the syntax but get confused about **when** each block runs. Use this mental model:

1. Control enters the `try` block and executes statements **top to bottom**.
2. If no exception occurs in the `try` block:
   - the `catch` blocks are skipped,
   - the `finally` block executes,
   - execution continues after the whole try/catch/finally structure.
3. If an exception occurs at some line inside `try`:
   - the remaining statements in `try` are skipped immediately,
   - Java searches catch blocks from **top to bottom** and selects the **first matching** one,
   - the selected `catch` runs (others are skipped),
   - then `finally` runs,
   - then execution continues after the structure.
4. If no `catch` matches the exception type:
   - `finally` still runs,
   - then the exception is **propagated** to the caller (the method exits abnormally).

Catch matching rule (very important). A catch matches if it has the same exception type or a **superclass** of the thrown exception. That is why catch order matters: specific first, generic last.

Return + finally. If you `return` from `try` or `catch`, Java still executes `finally` before returning.

Sample runs (what you should observe)

Case A: valid input

```
Enter a: 10
Enter b: 2
a/b = 5
Cleanup done (finally).
```

Case B: invalid number

```
Enter a: abc
Please enter valid integers.
Cleanup done (finally).
```

Case C: divide by zero

```
Enter a: 10
Enter b: 0
Cannot divide by zero.
Cleanup done (finally).
```

Note about closing `Scanner`. Closing a `Scanner` created from `System.in` also closes the underlying input stream. In a longer program that continues reading input later, you typically create one scanner and reuse it rather than repeatedly closing/reopening it.

6.7 `throw` vs `throws`

- `throw` actually throws an exception now.
- `throws` declares that a method may throw an exception (caller must handle/declare).

Example A: `throw` (explicitly throwing now)

Goal: stop the method immediately when an argument is invalid.

```java
static int divide(int a, int b) {
    if (b == 0) {
        throw new IllegalArgumentException("b must not be 0");
    }
    return a / b;
}
```

What happens at runtime?

- If `b == 0`, Java creates an `IllegalArgumentException` object and throws it.
- The method exits immediately; the `return` line is skipped.
- Control jumps to the nearest matching `catch` (if any), otherwise the program terminates with a stack trace.

When to use this pattern: for invalid arguments and broken assumptions (`IllegalArgumentException` is common).

Example B: `throws` (declaring a checked exception)

Goal: let the caller decide what to do if the operation fails.

```
static void pauseOneSecond() throws InterruptedException {
    Thread.sleep(1000); // may throw InterruptedException (checked)
}
```

Why `throws`? `InterruptedException` is a **checked** exception, so the compiler forces you to either:

- **catch** it, or
- **declare** it using `throws` in your own method signature.

Option 1: handle it (catch).

```
public static void main(String[] args) {
    try {
        pauseOneSecond();
        System.out.println("Done");
    } catch (InterruptedException ex) {
        System.out.println("Interrupted");
    }
}
```

Meaning: you handle the problem here (show message, cleanup, retry, etc.).

Option 2: propagate it (declare throws).

```
public static void main(String[] args) throws InterruptedException {
    pauseOneSecond();
    System.out.println("Done");
}
```

Meaning: you are not handling it here; you are pushing responsibility to your caller (for `main`, this means the JVM may print a stack trace if it happens).

Where should you use `throws`? Use `throws` when the method cannot handle the problem meaningfully and it is better to let a higher-level caller decide what to do.

6.8 Custom exception design (basic, but practical)

Create a custom exception when you want a clear domain-specific error message and type. Good custom exceptions:

- are named after the problem (`InvalidBookingException`),
- include a helpful message for debugging/user feedback,
- (optionally) wrap the original cause exception.

Checked or unchecked for custom exceptions?

- Use **unchecked** (`extends RuntimeException`) for validation and invalid arguments.
- Use **checked** (`extends Exception`) when callers should be forced to handle/recover.

Sources: Schildt, *Java: The Complete Reference* (12e) (nested/inner classes, exceptions, threads, I/O).; Oracle Java Tutorials: Nested Classes + Exceptions + Concurrency + Basic I/O (accessed February 27, 2026).; Java SE API docs (e.g., `Thread`, `AutoCloseable`) (accessed February 27, 2026).

7 Worked Example (tutorial): build a safe validation flow

We will design a small method that validates input and reports problems clearly.

Step 1: Create a small validation exception (unchecked)

```
class ValidationException extends RuntimeException {
    public ValidationException(String message) {
        super(message);
    }

    public ValidationException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

Why unchecked? If a user enters invalid input, we usually show a message and ask again. We don't want to clutter every method with mandatory `throws`.

Step 2: Write a parsing method that throws a clear exception

```
class Parsers {
    static int parsePositiveInt(String s) {
        if (s == null) throw new ValidationException("Input is required.");
        s = s.trim();
        if (s.isEmpty()) throw new ValidationException("Input cannot be empty.");

        try {
            int x = Integer.parseInt(s);
            if (x <= 0) throw new ValidationException("Number must be > 0.");
            return x;
        } catch (NumberFormatException ex) {
            throw new ValidationException("Not a valid integer: " + s, ex);
        }
    }
}
```

Notice:

- We validate **null** and **empty** before parsing.
- We catch `NumberFormatException` and throw a better message.
- We keep the original cause (`ex`) for debugging.

Step 3: Use it in `main` and handle only at the boundary

```java
import java.util.Scanner;

public class Demo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter number of tickets: ");
            int tickets = Parsers.parsePositiveInt(sc.nextLine());
            System.out.println("You booked " + tickets + " tickets.");
        } catch (ValidationException ex) {
            System.out.println("Input error: " + ex.getMessage());
        } finally {
            sc.close();
        }
    }
}
```

This is a clean pattern: validate/throw in low-level methods, and handle at the boundary (UI/CLI layer) where you can show messages.

8 Worked Example: custom checked exception (recoverable case)

```java
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

class BankAccount {
    private double balance;

    public BankAccount(double openingBalance) {
        if (openingBalance < 0) throw new IllegalArgumentException("openingBalance");
        balance = openingBalance;
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount <= 0) throw new IllegalArgumentException("amount");
        if (amount > balance) throw new InsufficientFundsException("Need " + amount + ",
            have " + balance);
        balance -= amount;
    }
}
```

```java
public class Demo {
    public static void main(String[] args) {
        BankAccount a = new BankAccount(100.0);
        try {
            a.withdraw(150.0);
        } catch (InsufficientFundsException e) {
            System.out.println("Withdraw failed: " + e.getMessage());
```

```
        }
    }
}
```

Here the caller can decide an alternate action: deposit more money, withdraw less, or show a message. That makes a checked exception reasonable.

9 In-Class Exercises (with brief solutions)

1. Name one checked and one unchecked exception.

   Answer: checked: `IOException`; unchecked: `NullPointerException`.

2. What is the difference between `throw` and `throws`?

   Answer: `throw` throws now; `throws` declares possibility in method signature.

3. When does `finally` execute?

   Answer: almost always (whether exception happens or not), used for cleanup.

10 Common bad practices (avoid)

- **Empty catch block**: hides bugs and makes debugging impossible.
- **Catching `Exception` everywhere**: can swallow unrelated problems.
- **Using exceptions for normal flow**: exceptions are slower and reduce clarity.
- **Throwing generic `Exception`**: prefer specific types with good messages.

11 Self-Study Practice (no solutions)

1. Write a method `parsePositiveInt(String s)` that throws `IllegalArgumentException` on invalid input.
2. Write a program that reads 2 integers and prints division result; handle divide-by-zero properly.
3. Design a custom exception `InvalidBookingException` for a booking system.

12 Summary (what to remember)

- Exceptions separate **normal flow** from **error handling**.
- Checked exceptions force callers to handle recoverable problems.
- Unchecked exceptions are used for invalid arguments and programming errors.
- Handle exceptions at boundaries (UI/CLI); throw/propagate from deeper logic.
- Learn to read stack traces: find your code line and the root cause.

13 Checkpoint Questions (with sample answers)

Q1. Why should we catch specific exceptions instead of `Exception`?

Sample answer: Catching specific exceptions helps you handle known cases correctly and avoids hiding unrelated bugs. Catching `Exception` broadly can swallow errors and make debugging hard.

Q2. What does checked exception mean?

Sample answer: A checked exception must be handled (caught) or declared with `throws`; otherwise the code will not compile.